Plan de Sprint — Projet Full-Stack

Spring Boot + Frontend · Groupe de 2 développeurs

Date limite : 28 mai 2026 | Durée estimée : 4 semaines | Riantsoa & Kevin

Principe de répartition

- **Riantsoa** = Authentification, Sécurité, Commandes + Frontend Auth & Espace Utilisateur
- **Kevin** = Produits, Catégories, Backoffice Admin + Frontend Public & Admin
- Chaque périmètre est **indépendant** : zéro blocage entre les deux
- Un contrat d'interface (DTO partagé) est défini au Jour 1

Stack technique

- **Backend :** Spring Boot 3, Spring Security, JWT, JPA/Hibernate, MySQL/PostgreSQL
- **Frontend :** Vue.js (recommandé) ou Angular
- **Outils :** Git (branches séparées), Postman, Maven/Gradle
- **Livraison :** /backend + /frontend dans un seul repo Git

Calendrier global

Setup	Sprint 1	Sprint 2	Sprint 3	Finalisation
Jour 1	Semaine 1–2	Semaine 2–3	Semaine 3–4	Derniers jours
Init projet & contrat	Backend core	Frontend pages	Intégration & tests	README, démo, rendu

Légende : BE Backend Spring Boot · FE Frontend · BOTH À faire ensemble (sync rapide)

Répartition détaillée des tâches

Riantsoa

Auth · Commandes · Espace Utilisateur

SETUP — JOUR 1

- BOTH Créer le repo Git, branches feature/dev-a et feature/dev-b
- BE Initialiser le projet Spring Boot (pom.xml : Security, JPA, JWT, Lombok, MySQL driver)
- BE Configurer application.properties (datasource, JWT secret, port)
- BE Créer la structure de packages : *entity, repository, service, controller, security, dto, exception*
- BOTH Définir ensemble les DTOs partagés (voir encadré ci-dessous)

Kevin

Produits · Catégories · Backoffice Admin

SETUP — JOUR 1

- BOTH Créer le repo Git, branches feature/dev-a et feature/dev-b
- FE Initialiser le projet Frontend (Vue.js recommandé) : structure de dossiers (components, services, views, models, router)
- FE Configurer le routing de base (public / utilisateur / admin)
- FE Configurer le fichier de configuration des URLs d'API (.env)
- BOTH Définir ensemble les DTOs partagés (voir encadré ci-dessous)

SPRINT 1 — BACKEND AUTH & SÉCURITÉ

- BE Entité **User** (id, username, email, password, roles, enabled, createdAt)
- BE Entité **Role** (enum : USER, ADMIN) — valeurs insérées en dur (data.sql)
- BE **UserRepository** (findByUsername, findByEmail)
- BE **JwtUtil** : génération, validation, extraction du username depuis le token
- BE **JwtAuthenticationFilter** : filtre HTTP pour valider le token à chaque requête
- BE **SecurityConfig** : règles (auth/**, public ; admin/**, ADMIN ; user/**, authenticated)
- BE **UserDetailsServiceImpl** : loadUserByUsername depuis la BDD
- BE **AuthController** : POST /api/auth/register & POST /api/auth/login (retourne JWT)
- BE Hachage du mot de passe avec BCryptPasswordEncoder
- BE Endpoint racine **GET /** → retourne une image (ressource statique)

SPRINT 1 — BACKEND COMMANDES

- BE Entité **Order** (id, user, orderDate, totalAmount, status, produit, quantite)
- BE Enum **OrderStatus** : PENDING, PROCESSING, SHIPPED, DELIVERED
- BE **OrderRepository** (findByUserId)
- BE **OrderService** : créer commande, calculer totalAmount, récupérer commandes de l'utilisateur
- BE **OrderController** : POST /api/orders, GET /api/orders/my-orders
- BE **AdminOrderEndpoint** dans AdminController : GET /api/admin/orders (liste toutes les commandes)
- BE Gestion des utilisateurs dans AdminController : GET /api/admin/users, PUT /api/admin/users/{id}

SPRINT 1 — BACKEND PRODUITS & CATÉGORIES

- BE Entité **Categorie** (id, name)
- BE Entité **Product** (id, name, description, price, category, stockQuantity, createdAt, lienImage)
- BE **CategorieRepository** et **ProductRepository** (fi fi
- BE **ProductService** : CRUD complet, méthode de recherche (nom + catégorie), pagination
- BE **ProductController** : GET /api/products (paginé), GET /api/products/{id}, GET /api/products/search?name=&category=
- BE Données de test : insérer 5–10 produits & 3–4 catégories via data.sql
- BE Configurer la gestion des ressources statiques (images produits dans /resources/static/images)

SPRINT 1 — BACKEND ADMINCONTROLLER

- BE **AdminController** — CRUD Produits : GET /api/admin/products, POST /api/admin/products, PUT /api/admin/products/{id}, DELETE /api/admin/products/{id}
- BE **AdminController** — CRUD Catégories : POST /api/admin/categories, PUT /api/admin/categories/{id}, DELETE /api/admin/categories/{id}
- BE Validation des DTOs côté backend (javax.validation : @NotBlank, @Positive, etc.)
- BE Gestion des erreurs globale : **GlobalExceptionHandler** (@ControllerA
- BE Configuration CORS (autoriser le port du frontend)
- BE Collection Postman : tester tous les endpoints admin & produit

SPRINT 2 — FRONTEND AUTH & ESPACE UTILISATEUR

- FE Initialiser le projet Vue.js (ou Angular) — routing, axios/http client, Pinia/store
- FE **AuthService** : appels login/register, stockage JWT en localStorage, helper isAuthenticated
- FE **Page Login** : formulaire email + mot de passe, gestion d'erreur, redirect après connexion
- FE **Page Register** : formulaire inscription, validation côté client
- FE **Route Guard** (NavigationGuard / CanActivate) pour les routes protégées
- FE Intercepteur HTTP : ajouter l'en-tête *Authorization: Bearer <token>* à chaque requête
- FE **Page Profil Utilisateur** : affichage des infos (username, email)
- FE **Page Historique des commandes** : liste GET /api/orders/my-orders
- FE **Page Passer une commande** : formulaire (choix produit + quantité), POST /api/orders
- FE Composant NavBar avec liens conditionnels (connecté / déconnecté / admin)

SPRINT 3 — INTÉGRATION & TESTS (PÉRIMÈTRE A)

- BE Tests Postman : register → login → passer commande → voir mes commandes
- FE Vérifier la gestion des 401/403 (redirection automatique vers login)
- FE Vérifier le flux complet : inscription → connexion → commande → historique
- BE Corriger les bugs remontés par Kevin (CORS, token expiré, etc.)
- BOTH Merge des branches, résolution des conflits

SPRINT 2 — FRONTEND PAGES PUBLIQUES

- FE **ProductService** : getAll(page), getById, search(name, category)
- FE **Page d'accueil** : grille de produits, pagination, barre de recherche, filtre par catégorie
- FE **Composant ProductCard** : image, nom, prix, catégorie, bouton "Voir le détail"
- FE **Page Détail Produit** : toutes les infos + image + stock + bouton "Commander"
- FE **Composant Pagination** réutilisable
- FE **Composant Filtres** : liste déroulante des catégories (GET /api/categories)

SPRINT 2 — FRONTEND BACKOFFICE ADMIN

- FE **Route Guard Admin** : redirection si rôle ≠ ADMIN
- FE **Layout Admin** : sidebar de navigation admin
- FE **Tableau de bord** : statistiques (nb produits, nb commandes, nb utilisateurs) via les endpoints existants
- FE **Page Gestion Produits** : tableau avec CRUD (modal création/édition, suppression avec confirmation)
- FE **Page Gestion Utilisateurs** : liste (GET /api/admin/users), modification de rôle (PUT /api/admin/users/{id})
- FE **Page Visualisation Commandes** : liste de toutes les commandes (GET /api/admin/orders)
- FE Composant de confirmation de suppression (modal générique)

SPRINT 3 — INTÉGRATION & TESTS (PÉRIMÈTRE B)

- BE Tests Postman : CRUD produits admin, recherche produits, liste catégories
- FE Vérifier le flux admin complet : login admin → dashboard → CRUD produit → gestion users
- FE Vérifier le flux public : accueil → recherche → détail produit
- FE Responsive design (mobile-friendly) sur les pages publiques
- BOTH Merge des branches, résolution des conflits

Contrat d'interfaces partagées — À définir Jour 1 (ensemble)

⚠ **Important :** Ces DTOs et URLs d'API doivent être définis et validés par les deux développeurs le premier jour. Ils permettent à Riantsoa et Kevin de travailler en parallèle sans se bloquer : Kevin peut mocker les endpoints de Riantsoa et vice-versa.

DTOs Java à créer en commun (package dto/)

```
// ── Auth ──────────────────────────────────────────
RegisterRequest  { username, email, password }
LoginRequest     { username, password }
AuthResponse     { token, username, roles }

// ── Produit ───────────────────────────────────────
ProductDTO       { id, name, description, price, category, stockQuantity, lienImage, createdAt }
CategoryDTO      { id, name }

// ── Commande ──────────────────────────────────────
CreateOrderRequest  { productId, quantite }
OrderResponse       { id, orderDate, totalAmount, status, productName, quantite }

// ── Utilisateur (admin) ───────────────────────────
UserDTO          { id, username, email, roles, enabled, createdAt }
UpdateUserRequest { roles, enabled }

// ── Pagination ────────────────────────────────────
PagedResponse<T> { content: List<T>, totalPages, totalElements, currentPage }
```

Points de synchronisation (sync rapides — 15 min max)

Quand	Objectif	Résultat attendu
Jour 1 — fin	Valider les DTOs, les URLs d'API et la structure du repo	Les deux développeurs connaissent exactement les contrats à respecter
Fin Semaine 1	Riantsoa montre login/register fonctionnel en Postman, Kevin montre CRUD produit	Les deux backends principaux sont testables indépendamment
Fin Semaine 2	Premier merge sur main, vérification que les deux backends coexistent sans erreur	Application Spring Boot unique qui démarre avec toutes les routes
Fin Semaine 3	Branchement Frontend → Backend réel (remplacer les mocks par les vrais appels)	Tous les flux end-to-end fonctionnent
Derniers jours	Revue finale ensemble, préparation démo, README	Projet prêt à soumettre

Pourquoi les tâches sont indépendantes

Backend : deux domaines séparés

Riantsoa gère *User, Role, Order*. Kevin gère *Product, Categorie*. Les packages ne se chevauchent pas. Riantsoa n'a pas besoin que les produits existent pour construire l'auth. Kevin peut créer de faux userId pour tester les commandes.

SecurityConfig = seul fichier partagé

Riantsoa écrit la *SecurityConfig* complète dès le Sprint 1. Kevin n'a qu'à la respecter (utiliser les bons en-têtes JWT dans Postman). Aucune modification de ce fichier n'est nécessaire de la part de Kevin.

Frontend : pages 100% séparées

Riantsoa travaille sur */login, /register, /profile, /orders*. Kevin travaille sur */, /products, /admin/***. Aucune page commune. Ils partagent uniquement le composant NavBar et l'intercepteur HTTP (défini par Riantsoa).

Stratégie Git recommandée

✓ Travaillez sur des branches séparées : **feature/dev-a** et **feature/dev-b**. Faites des merges vers **main** uniquement aux points de synchronisation validés ensemble. Utilisez des Pull Requests pour la revue croisée.

- [ ] Branch `feature/dev-a` créée et poussée
- [ ] Branch `feature/dev-b` créée et poussée
- [ ] Fichier `.gitignore` configuré (target/, node_modules/, .env)
- [ ] Structure /backend et /frontend à la racine
- [ ] Merge Semaine 2 validé ensemble
- [ ] Merge Semaine 3 validé ensemble

Checklist de livraison finale (les deux ensemble)

- [ ] Toutes les entités créées et migrées
- [ ] JWT : register + login fonctionnels
- [ ] Endpoints ADMIN protégés (test avec mauvais rôle)
- [ ] CRUD Produits & Catégories complet
- [ ] Création et liste de commandes fonctionnelles
- [ ] Page publique produits avec recherche + pagination
- [ ] Espace utilisateur (profil + commandes)
- [ ] Backoffice admin complet
- [ ] Collection Postman exportée
- [ ] README.md avec instructions d'installation
- [ ] 10 screenshots OU lien vidéo YouTube (unlisted)
- [ ] Repo Git public avec /backend et /frontend

Projet Full-Stack Spring Boot — ITIS / INGETIS / ING / BA DEV-DATA 3 · Date limite : 28 mai 2026
Document généré pour un groupe de 2 développeurs · Pour imprimer en PDF : Ctrl+P → Enregistrer en PDF